

Strutture dati

Array

Come usare gli array in C++ per memorizzare sequenze di valori dello stesso tipo.

Cos'è un array?

Immagina di dover memorizzare i voti di 30 studenti. Creeresti 30 variabili diverse? `voto1`, `voto2`, ... `voto30`? Sarebbe un disastro.

Un **array** è una fila di scatole numerate che contengono valori dello stesso tipo. Un solo nome, tanti valori. Puoi accedere a ciascuna scatola usando il suo numero (chiamato **indice**).

Dichiarare un array

```
tipo nome[dimensione];
```

```
int numeri[5];      // una fila di 5 numeri interi
double voti[10];    // una fila di 10 numeri decimali
char lettere[26];    // una fila di 26 caratteri
```

La dimensione deve essere un numero fisso, noto in anticipo (prima dell'esecuzione del programma).

Inizializzare un array

Puoi assegnare i valori subito alla dichiarazione, tra parentesi graffe:

```
int numeri[5] = {10, 20, 30, 40, 50};
double voti[4] = {8.5, 7.0, 9.0, 6.5};
char vocali[5] = {'a', 'e', 'i', 'o', 'u'};
```

Se non scrivi la dimensione, viene calcolata automaticamente dai valori:

```
int numeri[] = {10, 20, 30, 40, 50}; // dimensione 5, calcolata
automaticamente
```

Per inizializzare tutti gli elementi a zero:

```
int numeri[5] = {0}; // tutti e 5 a zero
int numeri2[5] = {}; // anche questo funziona in C++11
```

Se fornisci meno valori della dimensione, il resto viene azzerato:

```
int numeri[5] = {1, 2}; // diventa {1, 2, 0, 0, 0}
```

Accedere agli elementi

Ogni elemento ha un numero d'ordine chiamato **indice**. Attenzione: gli indici partono da **0**, non da 1.

```
int numeri[] = {10, 20, 30, 40, 50};
//indice:      0   1   2   3   4

cout << numeri[0] << endl; // 10 - primo elemento
cout << numeri[1] << endl; // 20
cout << numeri[4] << endl; // 50 - ultimo elemento
```

Per un array di dimensione **n**, gli indici vanno da **0** a **n-1**.

Modificare un elemento

```
int numeri[] = {10, 20, 30};
numeri[1] = 99;           // cambiamo il secondo elemento
cout << numeri[1] << endl; // 99
```

Pericolo: uscire dai limiti

In C++ non c'è alcun controllo automatico sui limiti. Se accedi a un indice che non esiste, il comportamento è imprevedibile e può corrompere la memoria del programma:

```
int numeri[5] = {1, 2, 3, 4, 5};
cout << numeri[10] << endl; // PERICOLO: indice fuori limiti
numeri[10] = 99;           // PERICOLO: potrebbe causare crash o dati
corrotti
```

Fai sempre attenzione che gli indici siano compresi tra `0` e `dimensione - 1`.

Scorrere un array con `for`

Il modo più comune per lavorare con tutti gli elementi di un array:

```
int numeri[] = {10, 20, 30, 40, 50};
int n = 5;

for (int i = 0; i < n; i++) {
    cout << numeri[i] << " ";
}
// Output: 10 20 30 40 50
```

Calcolare la dimensione automaticamente

Puoi ricavare il numero di elementi con `sizeof` :

```
int numeri[] = {10, 20, 30, 40, 50};  
// sizeof(numeri) = dimensione totale in byte dell'array  
// sizeof(numeri[0]) = dimensione in byte di un singolo elemento  
int dimensione = sizeof(numeri) / sizeof(numeri[0]);  
cout << dimensione << endl; // 5
```

Scorrere senza indice (range-based for)

In C++ moderno puoi scorrere tutti gli elementi in modo più semplice:

```
int numeri[] = {10, 20, 30, 40, 50};  
  
for (int n : numeri) {  
    cout << n << " ";  
}  
// Output: 10 20 30 40 50
```

Si legge: "per ogni `n` nell'array `numeri`".

Operazioni comuni

Trovare il valore massimo

```
int numeri[] = {3, 7, 1, 9, 4};
int n = 5;
int massimo = numeri[0]; // partiamo dal primo elemento

for (int i = 1; i < n; i++) {
    if (numeri[i] > massimo) {
        massimo = numeri[i]; // aggiorniamo il massimo se troviamo un
        valore più grande
    }
}

cout << "Massimo: " << massimo << endl; // 9
```

Calcolare somma e media

```
int numeri[] = {5, 10, 15, 20, 25};
int n = 5;
int somma = 0;

for (int i = 0; i < n; i++) {
    somma += numeri[i];
}

double media = (double)somma / n;
cout << "Somma: " << somma << endl; // 75
cout << "Media: " << media << endl; // 15
```

Esempio pratico

```
#include <iostream>
using namespace std;

int main() {
    const int N = 5;
    int voti[N];

    // Raccogliamo i voti dall'utente
    cout << "Inserisci " << N << " voti:" << endl;
    for (int i = 0; i < N; i++) {
        cout << "Voto " << (i + 1) << ": ";
        cin >> voti[i];
    }

    // Calcoliamo somma, massimo e minimo
    int somma = 0;
    int max = voti[0];
    int min = voti[0];

    for (int i = 0; i < N; i++) {
        somma += voti[i];
        if (voti[i] > max) max = voti[i];
        if (voti[i] < min) min = voti[i];
    }

    double media = (double)somma / N;

    cout << endl;
    cout << "Media: " << media << endl;
    cout << "Voto più alto: " << max << endl;
    cout << "Voto più basso: " << min << endl;

    return 0;
}
```

Stringhe C-style vs string

La differenza tra le stringhe in stile C (char array) e il tipo string del C++.

Le stringhe in stile C

In C++ esistono due modi diversi per lavorare con le parole e le frasi. Il primo è il modo "vecchio", ereditato dal linguaggio C. Il secondo è il modo moderno, molto più semplice.

Devi conoscere entrambi perché li incontrerai nel codice esistente, ma nella pratica ne userai quasi sempre uno solo.

I due metodi

1. **Stringhe C-style** — sono array di caratteri (`char[]`), ereditate dal linguaggio C. Più complicate da usare.
2. **`std::string`** — la stringa moderna del C++, molto più semplice e sicura.

Regola pratica: usa sempre `std::string` . Impara le stringhe C-style solo per capire il codice che le usa.

Le stringhe C-style: array di caratteri

Una stringa C-style è in realtà un array di `char` che termina con un carattere speciale: il carattere nullo `'\0'` . Questo terminatore dice "la stringa finisce qui".

```
char saluto[] = "Ciao";  
// Internamente memorizzata come: {'C', 'i', 'a', 'o', '\0'}  
// Occupa 5 byte (4 lettere + il terminatore)
```

Puoi accedere ai singoli caratteri con l'indice:

```
char nome[] = "Alice";  
cout << nome[0] << endl; // A  
cout << nome[1] << endl; // l
```

Le funzioni per le C-style strings

Per manipolare queste stringhe hai bisogno di funzioni speciali dalla libreria `<cstring>` :

```
#include <cstring>

char a[] = "Ciao";
char b[] = "Mondo";

cout << strlen(a) << endl;           // 4 – lunghezza della stringa (senza
'\0')
strcpy(a, "Hello");                  // copia "Hello" in a
strcat(a, " World");                 // concatena " World" ad a
cout << strcmp("abc", "abc");        // 0 – le due stringhe sono uguali
```

I problemi delle stringhe C-style

- Non puoi concatenarle con `+`
- Devi gestire la memoria manualmente
- È facile scrivere oltre i limiti del buffer (bug difficile da trovare)
- Il codice risultante è lungo e poco leggibile

```
char a[10] = "Ciao";
// ERRORE – non puoi fare a + " mondo" con le C-style strings
// Devi usare: strcat(a, " mondo");
```

Le stringhe moderne: `std::string`

`std::string` risolve tutti i problemi delle stringhe C-style. Per usarla, includi `<string>` :


```
#include <string>
using namespace std;

string nome = "Alice";
string cognome = "Bianchi";

// Concatenazione semplice con +
string nomeCompleto = nome + " " + cognome;

// Lunghezza con .length()
cout << nome.length() << endl;    // 5

// Confronto diretto con ==
if (nome == "Alice") {
    cout << "Ciao Alice!" << endl;
}
```

Tutto quello che con le C-style strings richiedeva funzioni speciali, con `std::string` si fa con operatori normali.

Confronto diretto

Operazione	C-style (char[])	std::string
Dichiarazione	<code>char s[20] = "ciao";</code>	<code>string s = "ciao";</code>
Lunghezza	<code>strlen(s)</code>	<code>s.length()</code>
Copia	<code>strcpy(dest, src)</code>	<code>dest = src</code>
Concatenazione	<code>strcat(a, b)</code>	<code>a + b</code>
Confronto	<code>strcmp(a, b) == 0</code>	<code>a == b</code>
Accesso a un carattere	<code>s[i]</code>	<code>s[i]</code>

Convertire tra i due tipi

A volte devi passare da uno all'altro (per esempio, alcune librerie richiedono il tipo C-style):

```
// Da string a char[] (C-style)
string str = "Ciao";
const char* cstr = str.c_str(); // converte in puntatore al C-string

// Da char[] a string – la conversione è automatica
char cArray[] = "Ciao";
string str2 = cArray;
```

Quando si usano le C-style strings

In pratica moderna, usa sempre `std::string`. Le C-style strings restano necessarie solo in casi particolari:

- Lavorare con librerie scritte in C che richiedono `const char*`
- Programmazione di sistema a basso livello
- Programmi dove ogni byte di memoria è importante

```
// Le librerie C spesso richiedono const char*
// Con std::string usi .c_str() per convertire
string nomefile = "dati.txt";
FILE* f = fopen(nomefile.c_str(), "r");
```

Esempio: confronto pratico

```
#include <iostream>
#include <string>
#include <cstring>
using namespace std;

int main() {
    // Approccio C-style – verboso e fragile
    char c_nome[50] = "Alice";
    char c_cognome[50] = "Bianchi";
    char c_completo[100];
    strcpy(c_completo, c_nome);    // copia il nome
    strcat(c_completo, " ");       // aggiunge uno spazio
    strcat(c_completo, c_cognome); // aggiunge il cognome
    cout << "C-style: " << c_completo << endl;

    // Approccio moderno con string – semplice e leggibile
    string s_nome = "Alice";
    string s_cognome = "Bianchi";
    string s_completo = s_nome + " " + s_cognome;
    cout << "string:  " << s_completo << endl;

    return 0;
}
```

Il codice con `std::string` è molto più semplice e difficile da sbagliare.

Iteratori in C++

Introduzione agli iteratori della STL in C++.

Cos'è un iteratore?

Gli algoritmi della STL (come `sort`, `find`, `count`) non lavorano direttamente con i contenitori. Lavorano con degli oggetti speciali chiamati **iteratori** che indicano "da dove a dove" operare.

Capire gli iteratori ti permette di usare tutti gli algoritmi della STL e di scorrere i contenitori in modi più sofisticati.

Cosa sono gli iteratori

Un iteratore è come un **segnalibro**: punta a un elemento del contenitore e può spostarsi all'elemento successivo (o precedente).

Pensa a un iteratore come a un dito che scorre su una lista: parte dall'inizio, si sposta verso destra elemento per elemento, e si ferma alla fine.

I metodi `begin()` e `end()`

Ogni contenitore STL ha due metodi fondamentali:

- `begin()` — restituisce un iteratore che punta al **primo** elemento
- `end()` — restituisce un iteratore che punta **dopo** l'ultimo elemento

```
vector<int> v = {10, 20, 30, 40, 50};

// begin() punta a 10
// end() punta a una posizione immaginaria dopo il 50
```

`end()` non punta a un elemento reale: è solo un segnaposto che indica "siamo arrivati alla fine".

Usare un iteratore

Dichiara un iteratore con `auto` (il compilatore capisce il tipo da solo):

```
vector<int> v = {10, 20, 30, 40, 50};

auto it = v.begin();           // it punta al primo elemento (10)
cout << *it << endl;          // 10 - il * "legge" il valore puntato

++it;                          // sposta l'iteratore al prossimo elemento
cout << *it << endl;          // 20

++it;
cout << *it << endl;          // 30
```

L'operatore `*` davanti all'iteratore "dereferenzia": legge il valore dell'elemento puntato.

Scorrere con un ciclo

```
vector<int> v = {10, 20, 30, 40, 50};

// Ciclo while con iteratore
auto it = v.begin();
while (it != v.end()) {
    cout << *it << " ";
    ++it; // sposta al prossimo
}
// Output: 10 20 30 40 50
```

```
// Ciclo for con iteratore – forma compatta
for (auto it = v.begin(); it != v.end(); ++it) {
    cout << *it << " ";
}
```

:::note Il range-based for (`for (int n : v)`) usa gli iteratori internamente. Quindi di solito non devi scrivere esplicitamente gli iteratori, ma capire come funzionano ti aiuta a usare la STL. :::

Iteratori che non modificano: `cbegin()` e `cend()`

Se vuoi scorrere gli elementi senza modificarli, usa `cbegin()` e `cend()` (la `c` sta per "constant"):

```
vector<int> v = {1, 2, 3};

for (auto it = v.cbegin(); it != v.cend(); ++it) {
    cout << *it << " ";
    // *it = 99; // ERRORE – con l'iteratore costante non puoi modificare
}
```

Scorrere al contrario: `rbegin()` e `rend()`

```
vector<int> v = {1, 2, 3, 4, 5};

for (auto it = v.rbegin(); it != v.rend(); ++it) {
    cout << *it << " ";
}
// Output: 5 4 3 2 1
```

`rbegin()` punta all'ultimo elemento, `rend()` punta "prima" del primo.

Iteratori e algoritmi STL

Gli iteratori sono il modo in cui colleghi i tuoi dati agli algoritmi:

```
#include <algorithm>
#include <vector>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina tutti gli elementi da begin a end
sort(v.begin(), v.end());

// Cerca il valore 5 e restituisce un iteratore all'elemento trovato
auto it = find(v.begin(), v.end(), 5);
if (it != v.end()) {
    // it - v.begin() calcola la posizione (distanza dall'inizio)
    cout << "Trovato 5 alla posizione " << (it - v.begin()) << endl;
}

// Rimuove l'elemento alla posizione 2
v.erase(v.begin() + 2);
```

Se `find` non trova il valore, restituisce `v.end()`. Ecco perché il controllo `if (it != v.end())` è importante.

Esempio pratico

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> numeri = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};

    // Stampa la lista originale usando un iteratore costante
    cout << "Originale: ";
    for (auto it = numeri.cbegin(); it != numeri.cend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // Ordiniamo il vector
    sort(numeri.begin(), numeri.end());
    cout << "Ordinato: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    // Stampiamo al contrario con l'iteratore inverso
    cout << "Inverso: ";
    for (auto it = numeri.rbegin(); it != numeri.rend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // unique rimuove i duplicati consecutivi – richiede il vector ordinato
    auto fine = unique(numeri.begin(), numeri.end());
    numeri.erase(fine, numeri.end()); // rimuoviamo i "resti" lasciati da
unique
    cout << "Senza duplicati: ";
    for (int n : numeri) cout << n << " ";
    cout << endl;

    return 0;
}
```

Output:

```
Originale: 3 1 4 1 5 9 2 6 5 3  
Ordinato: 1 1 2 3 3 4 5 5 6 9  
Inverso: 9 6 5 5 4 3 3 2 1 1  
Senza duplicati: 1 2 3 4 5 6 9
```

Array Multidimensionali

Come usare array a due o più dimensioni in C++.

Array con più dimensioni

Un array normale è come una fila di scatole. Ma a volte i dati sono naturalmente organizzati in una **griglia**: i voti di più studenti in più materie, le celle di una scacchiera, i pixel di un'immagine.

Un **array bidimensionale** è come una tabella con righe e colonne: hai bisogno di due numeri per identificare ogni cella.

Dichiarare un array 2D

```
tipo nome[righe][colonne];
```

```
int tabella[3][4]; // 3 righe, 4 colonne – 12 celle in totale
```

Inizializzare un array 2D

Puoi inizializzarlo riga per riga, usando parentesi graffe interne:


```
int matrice[2][3] = {  
    {1, 2, 3},    // riga 0  
    {4, 5, 6}     // riga 1  
};
```

Accedere agli elementi

Devi usare **due indici**: `[riga][colonna]` . Entrambi partono da 0.

```
int matrice[2][3] = {  
    {1, 2, 3},  
    {4, 5, 6}  
};  
  
cout << matrice[0][0] << endl; // 1 - riga 0, colonna 0  
cout << matrice[0][2] << endl; // 3 - riga 0, colonna 2  
cout << matrice[1][1] << endl; // 5 - riga 1, colonna 1
```

Immagina le coordinate come `[Y][X]` : prima la riga (verticale), poi la colonna (orizzontale).

Scorrere con due cicli annidati

Per visitare tutti gli elementi di una matrice, si usano due cicli `for` annidati: uno per le righe, uno per le colonne:

```
int matrice[3][3] = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};

for (int i = 0; i < 3; i++) {           // ciclo esterno: scorre le righe
    for (int j = 0; j < 3; j++) {       // ciclo interno: scorre le colonne
        della riga i
        cout << matrice[i][j] << "\t";
    }
    cout << endl; // vai a capo dopo ogni riga
}
```

Output:

```
1  2  3
4  5  6
7  8  9
```

Esempio: registro voti degli studenti

Un uso pratico: memorizzare i voti di più studenti in più materie.

```

#include <iostream>
using namespace std;

int main() {
    const int STUDENTI = 3;
    const int MATERIE = 4;

    // voti[studente][materia]
    int voti[STUDENTI][MATERIE] = {
        {8, 7, 9, 6},    // Alice
        {5, 8, 7, 9},    // Bob
        {9, 9, 8, 10}    // Carlo
    };

    string nomi[] = {"Alice", "Bob", "Carlo"};
    string materie[] = {"Matematica", "Italiano", "Inglese", "Storia"};

    // Stampa la tabella con le medie
    for (int s = 0; s < STUDENTI; s++) {
        cout << nomi[s] << ": ";
        int somma = 0;
        for (int m = 0; m < MATERIE; m++) {
            cout << voti[s][m] << " ";
            somma += voti[s][m];
        }
        double media = (double)somma / MATERIE;
        cout << "→ media: " << media << endl;
    }

    return 0;
}

```

Esempio: scacchiera

```
#include <iostream>
using namespace std;

int main() {
    char scacchiera[8][8];

    // Riempiamo la scacchiera alternando # e .
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            // Se la somma degli indici è pari, casella nera
            if ((i + j) % 2 == 0) {
                scacchiera[i][j] = '#';
            } else {
                scacchiera[i][j] = '.';
            }
        }
    }

    // Stampa la scacchiera
    for (int i = 0; i < 8; i++) {
        for (int j = 0; j < 8; j++) {
            cout << scacchiera[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

Array a tre dimensioni

Puoi avere anche array con tre o più dimensioni, anche se diventano difficili da visualizzare:

```
int cubo[2][3][4]; // 2 "strati", 3 righe, 4 colonne

// Accesso con tre indici
cubo[0][1][2] = 42;
```

Un array 3D è come un cubo: ha profondità, altezza e larghezza. Nella pratica, gli array 2D coprono la maggior parte dei casi.

Limitazioni degli array C++

Gli array tradizionali hanno alcuni svantaggi:

- La dimensione deve essere fissa e nota prima dell'esecuzione
- Non si "ricordano" quanti elementi contengono
- Non controllano se stai uscendo dai limiti

Per questi motivi, nelle applicazioni moderne si usa spesso `vector` (vedi la sezione apposita), che è più flessibile e sicuro.

Cenni alla STL

Panoramica della Standard Template Library del C++ con i contenitori più utili.

Cos'è la STL?

Quando programmi, ti ritrovi spesso ad avere gli stessi problemi: "devo memorizzare una lista di elementi", "devo associare nomi a valori", "devo gestire una coda". Risolverli da zero ogni volta sarebbe faticoso.

La **STL** (Standard Template Library) è una raccolta di "strumenti pronti all'uso" inclusa nel C++. Fornisce strutture dati già pronte (liste, dizionari, code...) e algoritmi già scritti (ordinamento, ricerca...).

Cosa contiene la STL

La STL ha tre componenti principali:

- **Contenitori:** strutture per memorizzare dati (`vector` , `map` , `set` , ...)
- **Algoritmi:** funzioni per operare sui dati (`sort` , `find` , `count` , ...)
- **Iteratori:** oggetti per scorrere i contenitori (trattati nella sezione apposita)

I contenitori principali

vector — Lista dinamica

Già trattato nella sezione dedicata. È il contenitore più usato in assoluto.

```
#include <vector>
vector<int> v = {1, 2, 3, 4, 5};
v.push_back(6);
```

map — Dizionario (chiave → valore)

Un `map` associa ogni chiave a un valore. È come un dizionario: cerchi una parola (chiave) e trovi la sua definizione (valore). Le chiavi sono mantenute in ordine alfabetico/numerico.

```

#include <map>
using namespace std;

map<string, int> eta;
eta["Alice"] = 16;
eta["Bob"] = 17;
eta["Carlo"] = 15;

// Leggi il valore associato a una chiave
cout << eta["Alice"] << endl;    // 16

// Scorri tutte le coppie
for (auto& coppia : eta) {
    // .first è la chiave, .second è il valore
    cout << coppia.first << ": " << coppia.second << endl;
}
// Output (in ordine alfabetico):
// Alice: 16
// Bob: 17
// Carlo: 15

// Controlla se una chiave esiste
if (eta.count("Alice") > 0) {
    cout << "Alice è nel registro" << endl;
}

// Rimuovi una coppia
eta.erase("Bob");

```

unordered_map — Dizionario veloce

Come `map`, ma non mantiene l'ordine delle chiavi. In cambio è più veloce:

```
#include <unordered_map>
using namespace std;

unordered_map<string, int> contaParole;
contaParole["ciao"]++; // se la chiave non esiste, viene creata con
valore 0
contaParole["mondo"]++;
contaParole["ciao"]++;

for (auto& p : contaParole) {
    cout << p.first << ": " << p.second << " volte" << endl;
}
```

Usa `map` se ti serve l'ordine, `unordered_map` se ti serve la velocità.

set — Insieme (senza duplicati)

Un `set` memorizza elementi unici, in ordine. Se inserisci un valore già presente, non cambia nulla.

```
#include <set>
using namespace std;

set<int> numeri = {5, 2, 8, 2, 1, 9, 1}; // i duplicati vengono ignorati
// numeri contiene: {1, 2, 5, 8, 9}

numeri.insert(3); // aggiunge 3
numeri.erase(8); // rimuove 8

if (numeri.count(5) > 0) {
    cout << "5 è nel set" << endl;
}

for (int n : numeri) cout << n << " ";
// Output: 1 2 3 5 9
```

list — Lista collegata

Una `list` è efficiente quando aggiungi e rimuovi elementi in mezzo alla lista molto spesso:


```
#include <list>
using namespace std;

list<int> l = {1, 2, 4, 5};
l.push_front(0); // aggiunge all'inizio
l.push_back(6);  // aggiunge alla fine
l.pop_front();   // rimuove il primo

for (int n : l) cout << n << " ";
```

Per la maggior parte dei casi usa `vector`. Usa `list` solo se fai molte inserzioni/rimozioni nel mezzo.

stack — Pila (ultimo entrato, primo uscito)

Una pila (stack) funziona come una pila di piatti: metti sempre sopra, e prendi sempre dall'alto. L'ultimo elemento aggiunto è il primo a uscire (LIFO: Last In, First Out).

```
#include <stack>
using namespace std;

stack<int> pila;
pila.push(1); // metti 1 in cima
pila.push(2); // metti 2 sopra all'1
pila.push(3); // metti 3 sopra al 2

cout << pila.top() << endl; // 3 - elemento in cima
pila.pop();                 // rimuovi l'elemento in cima
cout << pila.top() << endl; // 2
```

queue — Coda (primo entrato, primo uscito)

Una coda (queue) funziona come la cassa al supermercato: il primo ad arrivare è il primo ad essere servito (FIFO: First In, First Out).

```
#include <queue>
using namespace std;

queue<string> coda;
coda.push("Alice");    // Alice arriva per prima
coda.push("Bob");
coda.push("Carlo");

cout << coda.front() << endl; // "Alice" – la prima ad essere servita
coda.pop();              // Alice viene servita e va via
cout << coda.front() << endl; // ora tocca a "Bob"
```

Riepilogo: quale contenitore usare?

Contenitore	Quando usarlo
<code>vector</code>	Uso generale, accesso per posizione
<code>map</code>	Associare chiavi a valori, con ordine
<code>unordered_map</code>	Associare chiavi a valori, più veloce
<code>set</code>	Memorizzare valori unici
<code>list</code>	Molte inserzioni/rimozioni in mezzo
<code>stack</code>	Algoritmi che tornano indietro (backtracking)
<code>queue</code>	Code di elaborazione

Gli algoritmi della STL

Con `#include <algorithm>` hai accesso a molti algoritmi già pronti:

```

#include <algorithm>
#include <vector>
#include <numeric>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina in ordine crescente
sort(v.begin(), v.end());

// Cerca se un valore è presente (richiede il vector ordinato)
bool trovato = binary_search(v.begin(), v.end(), 5);

// Conta quante volte appare un valore
int quanti = count(v.begin(), v.end(), 3);

// Trova il massimo e il minimo
auto massimo = *max_element(v.begin(), v.end());
auto minimo = *min_element(v.begin(), v.end());
cout << "Max: " << massimo << ", Min: " << minimo << endl;

// Somma tutti gli elementi (da <numeric>)
int somma = accumulate(v.begin(), v.end(), 0);

```

Vettori (vector)

Come usare il contenitore vector della STL in C++.

Cos'è un vector?

Un array normale ha un problema fondamentale: la dimensione deve essere decisa prima di eseguire il programma. Se non sai in anticipo quanti dati avrai, sei bloccato.

Il **vector** risolve questo problema: è come un array, ma **cambia dimensione automaticamente** mentre il programma gira. Aggiungi elementi e lui cresce. Rimuovili e lui si riduce.

Per usarlo, includi `<vector>` :

```
#include <vector>
using namespace std;
```

Creare un vector

```
// Vector vuoto – aggiungeremo elementi dopo
vector<int> v1;

// Vector con 5 elementi, tutti a zero
vector<int> v2(5);           // {0, 0, 0, 0, 0}

// Vector con valori già impostati
vector<int> v3 = {1, 2, 3, 4, 5};

// Vector con 4 elementi, tutti uguali a 7
vector<int> v4(4, 7);        // {7, 7, 7, 7}

// Vector di stringhe
vector<string> nomi = {"Alice", "Bob", "Carlo"};
```

Il tipo tra `<>` indica cosa contiene il vector. `vector<int>` contiene interi, `vector<string>` contiene stringhe, e così via.

Accedere agli elementi

Come gli array, si usa l'indice `[]` (parte da 0):

```
vector<int> v = {10, 20, 30, 40, 50};

cout << v[0] << endl;    // 10 – primo elemento
cout << v[2] << endl;    // 30
cout << v[4] << endl;    // 50 – ultimo elemento

// Modifica di un elemento
v[1] = 99;
cout << v[1] << endl;    // 99
```

Per un accesso più sicuro (il programma ti avvisa se esci dai limiti), usa `.at()` :

```
cout << v.at(2) << endl;    // 30
// v.at(10);                // lancia un errore se 10 è fuori range
```

Aggiungere e rimuovere elementi

```
vector<int> v;

// Aggiunge in coda – il vector cresce automaticamente
v.push_back(10);
v.push_back(20);
v.push_back(30);
// v è ora {10, 20, 30}

// Rimuove l'ultimo elemento
v.pop_back();
// v è ora {10, 20}
```

Informazioni sul vector

```
vector<int> v = {1, 2, 3, 4, 5};

cout << v.size() << endl;    // 5 – quanti elementi contiene
cout << v.empty() << endl;  // false (0) – è vuoto?

v.pop_back();
cout << v.size() << endl;    // 4

v.clear();                   // svuota completamente il vector
cout << v.size() << endl;    // 0
cout << v.empty() << endl;  // true (1)
```

Scorrere un vector

Con indice (come un array):

```
vector<int> v = {10, 20, 30, 40, 50};

for (int i = 0; i < v.size(); i++) {
    cout << v[i] << " ";
}
```

Con range-based for (il modo preferito — più semplice):

```
for (int n : v) {
    cout << n << " ";
}
```

Per modificare gli elementi mentre scorri, usa `&` :

```
// Moltiplica ogni elemento per 2
for (int& n : v) {
    n *= 2;
}
```

Primo e ultimo elemento

```
vector<int> v = {10, 20, 30};

cout << v.front() << endl;    // 10 - primo elemento
cout << v.back() << endl;     // 30 - ultimo elemento
```

Inserire e rimuovere in posizioni specifiche

```
vector<int> v = {1, 2, 4, 5};

// Inserisce 3 alla posizione 2 (tra il 2 e il 4)
v.insert(v.begin() + 2, 3);
// v è ora {1, 2, 3, 4, 5}

// Rimuove l'elemento alla posizione 1 (il 2)
v.erase(v.begin() + 1);
// v è ora {1, 3, 4, 5}
```

Ordinare un vector

Per ordinare, includi anche `<algorithm>` :

```
#include <algorithm>

vector<int> v = {5, 2, 8, 1, 9, 3};

sort(v.begin(), v.end());           // ordine crescente → {1, 2, 3, 5, 8, 9}
sort(v.begin(), v.end(), greater<int>()); // ordine decrescente → {9, 8, 5, 3, 2, 1}
```

Esempio pratico: raccolta voti interattiva

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<int> voti;
    int voto;

    cout << "Inserisci i voti (0 per terminare):" << endl;
    while (true) {
        cin >> voto;
        if (voto == 0) break;

        if (voto >= 1 && voto <= 10) {
            voti.push_back(voto); // aggiungiamo il voto al vector
        } else {
            cout << "Voto non valido, ignorato." << endl;
        }
    }

    if (voti.empty()) {
        cout << "Nessun voto inserito." << endl;
        return 0;
    }

    // Calcoliamo le statistiche
    int somma = 0;
    for (int v : voti) somma += v;
    double media = (double)somma / voti.size();

    // Ordiniamo per trovare min e max
    sort(voti.begin(), voti.end());

    cout << "\nNumero di voti: " << voti.size() << endl;
    cout << "Media: " << media << endl;
    cout << "Minimo: " << voti.front() << endl; // primo dopo ordinamento
    cout << "Massimo: " << voti.back() << endl; // ultimo dopo
    ordinamento

    cout << "Voti ordinati: ";
    for (int v : voti) cout << v << " ";
    cout << endl;
}
```



```
} return 0;
```